Aprendizagem de Máquina II — Teste 02 (RNN, LSTM e GRU)

Respostas às questões do notebook `Lab_Teste02_RNN_LSTM_GRU.ipynb`

Aluno: Bruno Araujo

---

1. Dataset — avaliações e classes

Questão 1

Entrada: cada avaliação é uma sequência de IDs inteiros de palavras, com vocabulário limitado às 5.000 mais frequentes (`V = 5000`). Depois do padding e do truncamento, cada sequência tem tamanho fixo `T = 120`. **Rótulos:** `0` é avaliação negativa e `1` é positiva.

Por que a ordem importa: o sentido depende da posição das palavras. "*not good*" e "*good*" usam quase as mesmas palavras e significam o contrário. Em "*I expected it to be bad, but it was great*", a conclusão vem no final e muda tudo. Um "saco de palavras" perde essa informação. Já a RNN lê token a token e atualiza o estado h_t usando o contexto anterior.

Questão 2

Partição	Instâncias	Classe 0	Classe 1
Treinamento	10.000	5.000	5.000
Validação	2.000	1.000	1.000
Teste	2.000	1.000	1.000

Mesmas partições: a comparação precisa ser justa, ou seja, a única variável deve ser a arquitetura recorrente. Com partições diferentes, um modelo poderia receber exemplos mais fáceis ou mais difíceis, e a diferença de desempenho viria dos dados e não da RNN, LSTM ou GRU. Além disso, a validação sai só da base de treino e o teste da base de teste, o que evita vazamento de dados.

---

2. Sequências, timesteps e embedding

Questão 3

- `X_train.shape = (10000, 120)`: 10.000 avaliações, cada uma com 120 IDs (um por timestep).
- `vetores_demo.shape = (2, 120, 24)`: 2 avaliações, 120 timesteps, e cada token virou um vetor de 24 dimensões.

Diferença entre os três hiperparâmetros:

- **`T = 120`:** comprimento da sequência, ou seja, quantos passos de tempo a rede processa.
- **`F = 24`:** dimensão de cada entrada x_t (tamanho do vetor de embedding de cada token).

- `H = 16`**:** número de neurônios da camada recorrente, que é a dimensão do estado oculto h_t (e de c_t na LSTM).

Questão 4

- **Exemplo curto (4 → 120):** houve **padding**. Os 4 IDs `[4, 18, 52, 97]` ficaram no início e os 116 restantes foram preenchidos com `0` no final (`padding="post"`).
- **Exemplo longo (130 → 120):** houve **truncamento**. Os 10 primeiros IDs (4 a 13) foram descartados; a sequência passa a começar em 14 e termina em 133 (`truncating="pre"`).

O que se perde numa avaliação real: o **início** do texto (introdução, contexto, primeira impressão e o token `[INICIO]`). Isso é relevante porque a mediana das avaliações é de 176 tokens, maior que `T`, então mais da metade delas é truncada.

Questão 5

Função do Embedding: os IDs são apenas índices arbitrários. O ID 50 não é "maior" nem "parecido" com o ID 51. O `Embedding` é uma tabela de consulta treinável (5.000 × 24) que transforma cada ID num vetor denso. Durante o treinamento, palavras com papel parecido na tarefa (por exemplo, "*great*" e "*excellent*") tendem a ficar com vetores próximos. Com `mask_zero=True`, os zeros do padding são ignorados pela camada recorrente.

Por que validação e teste não ajustam os pesos: seria **vazamento de dados**. A validação serve para selecionar o modelo (no caso, o Early Stopping) e o teste estima a generalização em dados nunca vistos. Se essas partições ajustassem o embedding, as métricas ficariam otimistas e não representariam o desempenho real.

---

3. Estados produzidos pelas três arquiteturas

Questão 6

A dimensão `(2, 120, 16)` significa: **2** avaliações (batch), **120** timesteps e **16** valores do estado oculto h_t em cada passo. A sequência inteira de estados aparece porque `return_sequences=True` devolve h_t em todos os passos, e não só o último.

Os pesos não mudam entre os timesteps. Os mesmos W_x, W_h e `b` são reutilizados em todos os passos (compartilhamento de parâmetros, a rede "desenrolada" no tempo).

Questão 7

Arquitetura	Estados transportados
RNN simples	h_t
LSTM	h_t e c_t
GRU	h_t (que acumula as duas funções)

Na LSTM:

- c_t **(estado da célula):** é a memória de longo prazo. É atualizada de forma aditiva e controlada pelas portas ($f \odot c_{t-1} + i \odot g$). Isso facilita o fluxo do gradiente e atenua o problema do

gradiente que desaparece.
- h_t **(estado oculto):** é a memória de curto prazo, a saída "exposta" do passo. É uma versão filtrada da célula, $o_t \odot \tanh(c_t)$, e é ela que vai para o próximo passo e para a camada `Dense`.

---

4. Portas e atualização da memória

Questão 8

Fórmula: $c_t = f_t \odot c_{t-1} + i_t \odot g_t$

Neurônio	Memória preservada ($f \odot c_{t-1}$)	Informação candidata ($i \odot g$)	c_t
1	0,25 × 0,8 = **0,200**	0,60 × (−0,5) = **−0,300**	**−0,100**
2	0,90 × (−0,4) = **−0,360**	0,20 × 0,7 = **0,140**	**−0,220**

Resultado: c_t = [−0,100, −0,220]

Interpretação:

- **Neurônio 1:** esqueceu quase toda a memória anterior (`f = 0,25`) e foi dominado pela informação nova.
- **Neurônio 2:** preservou a maior parte da memória (`f = 0,9`) e recebeu pouca informação nova (`i = 0,2`).

Complemento: $h_t = o \odot \tanh(c_t)$ = [0,8 × tanh(−0,1), 0,4 × tanh(−0,22)] ≈ [−0,080, −0,087].

Questão 9

- `f` **(forget gate):** decide quanto da memória anterior c_{t-1} é mantido.
- `i` **(input gate):** decide quanto da informação candidata `g` é escrito na célula.
- `o` **(output gate):** decide quanto de $\tanh(c_t)$ é exposto em h_t.

`g` **não é uma quarta porta.** As portas usam sigmoide, com valores entre 0 e 1, e funcionam como filtros multiplicativos ("quanto deixar passar"). Já `g` usa tanh, com valores entre −1 e 1, e é o **conteúdo** candidato, não um controle. Ela tem seu próprio conjunto de pesos, e é por isso que a LSTM tem 4 conjuntos (questão 11).

Questão 10

Fórmula: $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$

Neurônio	$z \odot h_{t-1}$	$(1 - z) \odot \tilde{h}$	h_t
1	0,80 × 0,60 = **0,480**	0,20 × (−0,20) = **−0,040**	**0,440**
2	0,25 × (−0,40) = **−0,100**	0,75 × 0,50 = **0,375**	**0,275**

Resultado: h_t = [0,440, 0,275]

Interpretação:

- **Neurônio 1:** `z = 0,8`, então preservou o estado anterior.

- **Neurônio 2:** `z = 0,25`, então foi dominado pelo candidato.

Diferença entre as portas:

- **Update gate (`z`):** atua na **saída final**. Faz uma "mistura" entre manter o estado antigo e adotar o novo; faz o papel conjunto de forget e input gate da LSTM.
- **Reset gate (`r`):** atua **antes**, no cálculo do candidato $\tilde{h}_t$. Decide quanto de h_{t-1} é usado para gerá-lo. Aqui, `r = 0,1` no neurônio 1 quer dizer que o candidato praticamente ignorou o passado; `r = 0,9` no neurônio 2 quer dizer que o passado foi bem usado. Como $\tilde{h}$ já foi fornecido, `r` não entra diretamente nesta conta.

---

5. Parâmetros treináveis — RNN, LSTM e GRU

Questão 11

RNN simples:

- Pesos de entrada (W_x): F × H = 24 × 16 = **384**
- Pesos recorrentes (W_h): H × H = 16 × 16 = **256**
- Biases: H = **16**
- **Total = 656 ✓** (formas `(24,16)`, `(16,16)`, `(16,)`)

LSTM (4 conjuntos: f, i, o, g):

- Pesos de entrada: 4 × 384 = **1.536** (forma `(24, 64)`)
- Pesos recorrentes: 4 × 256 = **1.024** (forma `(16, 64)`)
- Biases: 4 × 16 = **64**
- **Total = 2.624 ✓** (ou seja, 4 × 656)

Se T passasse de 120 para 240, os totais não mudariam. Os pesos são compartilhados entre os timesteps, e o número de parâmetros depende só de `F` e `H`. Aumentar `T` só aumenta o custo de processamento (mais passos).

Questão 12

GRU do Keras (`reset_after=True`), 3 conjuntos:

- Pesos de entrada: 3 × 24 × 16 = **1.152** (forma `(24, 48)`)
- Pesos recorrentes: 3 × 16 × 16 = **768** (forma `(16, 48)`)
- Biases: 2 × 3 × 16 = **96** (forma `(2, 48)`)
- **Total = 2.016 ✓**

Três conjuntos com só duas portas: além das portas `z` e `r`, o **estado candidato** $\tilde{h}_t$ também tem seus próprios pesos (mesma lógica do `g` na LSTM). Ficam então 2 portas + 1 candidato.

Diferença nos biases:

- A LSTM tem **um** vetor de bias por conjunto: 4 × 16 = 64.
- A GRU com `reset_after=True` aplica `r` **depois** da multiplicação recorrente ($r \odot (W_h h_{t-1} + b_h)$). Por isso ela precisa de **dois** biases por conjunto, um para a parte de entrada e outro para a parte recorrente: 2 × 48 = 96.

Mesmo assim, a GRU é mais leve que a LSTM (2.016 contra 2.624) por ter um conjunto a menos.

---

6. Treinamento comparativo

Questão 13

- **Hiperparâmetros de arquitetura:** `V = 5000`, `T = 120`, `F = 24`, `H = 16`, tipo da camada recorrente (SimpleRNN, LSTM ou GRU), `mask_zero=True`, padding `post` e truncamento `pre`, saída `Dense(1, sigmoid)`, `reset_after=True` na GRU.
- **Hiperparâmetros de treinamento:** otimizador Adam, learning rate 0,001, `batch_size = 128`, máximo de 8 épocas, perda `binary_crossentropy`, EarlyStopping (monitorando `val_loss`, `patience = 2`, `restore_best_weights`), `shuffle`, semente 42.
- **Parâmetros aprendidos:** matriz do Embedding (5.000 × 24 = 120.000), W_x, W_h e biases da camada recorrente, e pesos e bias da `Dense` (16 + 1 = 17). Por exemplo, RNN: 120.000 + 656 + 17 = 120.673 ✓.

Por que as épocas variam: o Early Stopping depende da trajetória da `val_loss` de cada modelo. Cada arquitetura atinge o mínimo num momento diferente e para 2 épocas depois:

Modelo	Melhor época	Épocas executadas
RNN	3	5
LSTM	2	4
GRU	2	4

Questão 14

Modelo	Melhor época	Menor `val_loss`	`val_accuracy`
RNN	3	0,4206	0,817
LSTM	**2**	**0,3838**	**0,828**
GRU	2	0,4006	0,818

A LSTM teve a menor loss de validação na melhor época.

Há sinais de overfitting nos três modelos. A loss de treinamento continua caindo (por volta de 0,59–0,68 até 0,24–0,27), enquanto a de validação volta a subir depois da melhor época:

- **LSTM:** 0,384 → 0,389 → 0,495.
- **GRU:** 0,401 → 0,496 → 0,475.
- **RNN:** 0,421 → 0,434 → 0,424 (overfitting mais leve).

A distância entre a acurácia de treino (~0,90–0,91) e a de validação (~0,80) cresce nas últimas épocas. O Early Stopping limitou o efeito ao restaurar os pesos da melhor época.

---

7. Avaliação em teste e interpretação de previsões

Questão 15

Modelo	Acurácia teste	Loss teste
RNN	0,825	0,4116
LSTM	**0,835**	0,3880
GRU	0,834	**0,3767**

Os três ficaram muito acima da referência de 50% (cerca de +33 pontos percentuais), então aprenderam padrões reais. **Na acurácia, a LSTM foi a melhor nesta execução**, mas por apenas 0,001 sobre a GRU, que teve a menor loss de teste. A RNN ficou cerca de 1 ponto atrás.

Por que isso não basta para uma conclusão geral:

- Foi uma única semente e uma única partição.
- Diferenças de cerca de 1 ponto percentual correspondem a só ~20 avaliações em 2.000.
- O resultado depende das escolhas de `T`, `H`, learning rate e do número de épocas.
- Com `T = 120`, as sequências são relativamente curtas, o que reduz a vantagem da LSTM e da GRU em dependências longas.

Uma conclusão sólida exigiria repetir o experimento com várias sementes ou validação cruzada e comparar média e desvio.

Questão 16

- **Previsões semelhantes, Ex. 5 (real = 1):** RNN 0,882, LSTM 0,868, GRU 0,892. Os três acertam com alta confiança.
- **Previsões diferentes, Ex. 7 (real = 1):** RNN 0,174 (errou, prevendo negativo com cerca de 83% de confiança), LSTM 0,750 e GRU 0,643 (acertaram).
- **Outro caso divergente, Ex. 3 (real = 0):** RNN 0,550 e LSTM 0,580 erraram por pouco, e só a GRU acertou (0,342).

Probabilidades altas não garantem acerto. A sigmoide indica a confiança do modelo, não a verdade. No Ex. 7, a RNN estava bastante "segura" da classe negativa e errou. Isso pode ter relação com o truncamento: o trecho começa no meio de uma frase e tem muitos `[DESCONHECIDO]`, ou seja, o modelo viu informação incompleta. Valores perto de 0,5 (Ex. 3) indicam incerteza, e é ali que os modelos mais divergem.